MÁSTER EN DATA SCIENCE E INTELIGENCIA ARTIFICIAL

SQL vs NoSQL

Análisis Comparativo de Ventas en DataMart Global

Caso práctico evaluable · Tema 6 — Bases de Datos NoSQL

Autor: **Claudio Abreu**

MySQL 8 · MongoDB · Aggregation Pipeline

EBIS Business Techschool

Contexto

Como Data Analyst en **DataMart Global** (e-commerce de tecnología y hogar), se solicita reproducir dos análisis idénticos sobre las mismas ventas en dos motores distintos —MySQL (relacional) y MongoDB (documental)— para evaluar diferencias de complejidad, rendimiento y mantenibilidad. Ambas bases contienen la misma información: 15 clientes, 15 productos, 30 pedidos y 52 líneas de detalle.

Análisis 1 · Top 10 clientes por gasto total

> **Objetivo:** identificar los 10 clientes que más han gastado, mostrando nombre, país, número de pedidos y monto total, ordenado de mayor a menor.

Solución en SQL (JOIN + agregación)

```sql
SELECT
    c.nombre,
    c.pais,
    COUNT(o.id_pedido) AS total_pedidos,
    ROUND(SUM(o.total), 2) AS monto_total
FROM customers c
JOIN orders o ON c.id_cliente = o.id_cliente
GROUP BY c.id_cliente, c.nombre, c.pais
ORDER BY monto_total DESC
LIMIT 10;
```

Solución en MongoDB (aggregation pipeline)

```javascript
db.orders.aggregate([
  { $lookup: { from: "customers", localField: "id_cliente",
               foreignField: "id_cliente", as: "cliente" } },
  { $unwind: "$cliente" },
  { $group: {
      _id: { id: "$id_cliente", nombre: "$cliente.nombre", pais: "$cliente.pais" },
      total_pedidos: { $sum: 1 },
      monto_total: { $sum: "$total" } } },
  { $sort: { monto_total: -1 } },
  { $limit: 10 },
  { $project: { _id: 0, nombre: "$_id.nombre", pais: "$_id.pais",
                total_pedidos: 1, monto_total: { $round: ["$monto_total", 2] } } }
])
```

*Nota técnica: el **$unwind** es necesario porque **$lookup** devuelve siempre un array; al aplanarlo, `$cliente.nombre` apunta a un objeto y no a una lista.*

Resultado (idéntico en ambos motores):

#	Cliente	País	Pedidos	Monto total (€)
1	Lars Anderson	Suecia	2	3.699,95
2	María García	España	4	3.599,95
3	Ana Silva	Brasil	3	3.349,94

4	John Smith	USA	3	3.299,97
5	Klaus Mueller	Alemania	2	3.299,95
6	Emma Johnson	UK	2	3.049,96
7	Pierre Dubois	Francia	3	2.899,95
8	Yuki Tanaka	Japón	2	2.749,96
9	Hans Schmidt	Alemania	1	2.299,96
10	Sofia Rossi	Italia	2	1.979,96

Interpretación de negocio: el cliente de mayor valor (Lars Anderson) alcanza el primer puesto con solo 2 pedidos, mientras que María García, con el doble de pedidos (4), gasta menos. Esto indica que **el ticket medio importa más que la frecuencia**: conviene segmentar a los clientes de alto valor para programas de fidelización premium, y no asumir que quien compra más veces es quien más aporta.

Análisis 2 · Top 3 productos más vendidos por categoría

> **Objetivo:** identificar los 3 productos más vendidos (por unidades) dentro de cada categoría, mostrando categoría, producto, cantidad total e ingresos generados.

Solución en SQL (CTE + ROW_NUMBER)

```sql
WITH ranked AS (
  SELECT
      p.categoria,
      p.nombre_producto,
      SUM(od.cantidad) AS cantidad_total,
      ROUND(SUM(od.cantidad * od.precio_unitario), 2) AS ingresos_totales,
      ROW_NUMBER() OVER (
          PARTITION BY p.categoria
          ORDER BY SUM(od.cantidad) DESC, SUM(od.cantidad * od.precio_unitario) DESC
      ) AS rn
  FROM products p
  JOIN order_details od ON p.id_producto = od.id_producto
  GROUP BY p.id_producto, p.categoria, p.nombre_producto
)
SELECT categoria, nombre_producto, cantidad_total, ingresos_totales
FROM ranked
WHERE rn <= 3
ORDER BY categoria, cantidad_total DESC;
```

*El **PARTITION BY** reinicia la numeración en cada categoría, de modo que `rn <= 3` recorta el top 3 por grupo y no el top 3 global. Se añade un segundo criterio de orden (ingresos) como **desempate** ante cantidades iguales.*

Solución en MongoDB ($push + $slice)

```javascript
db.order_details.aggregate([
  { $lookup: { from: "products", localField: "id_producto",
               foreignField: "id_producto", as: "producto" } },
  { $unwind: "$producto" },
  { $group: {
      _id: { categoria: "$producto.categoria", nombre: "$producto.nombre_producto" },
      cantidad_total: { $sum: "$cantidad" },
      ingresos_totales: { $sum: { $multiply: ["$cantidad", "$precio_unitario"] } } } },
  { $sort: { "_id.categoria": 1, cantidad_total: -1, ingresos_totales: -1 } },
  { $group: {
      _id: "$_id.categoria",
      productos: { $push: { nombre: "$_id.nombre", cantidad_total: "$cantidad_total",
                            ingresos_totales: { $round: ["$ingresos_totales", 2] } } } } },
  { $project: { categoria: "$_id", top3: { $slice: ["$productos", 3] } } },
  { $unwind: "$top3" },
  { $sort: { categoria: 1, "top3.cantidad_total": -1 } }
])
```

*Estrategia equivalente sin función ventana: el primer **$group** agrega por producto; el segundo apila los productos ya ordenados con **$push** y **$slice** recorta a 3. El **$sort** previo es imprescindible para que el array entre ordenado.*

Resultado (idéntico en ambos motores, con desempate por ingresos):

Categoría	Producto	Uds. vendidas	Ingresos (€)
Tecnología	Laptop Dell XPS 13	8	10.399,92

Tecnología	iPhone 14 Pro	8	8.799,92
Tecnología	Samsung Galaxy S23	5	4.499,95
Hogar	Lámpara de Pie LED	9	1.169,91
Hogar	Sofá Moderno 3 Plazas	7	5.599,93
Hogar	Mesa de Comedor Roble	3	1.649,97
Electrodom.	Cafetera Nespresso	9	1.799,91
Electrodom.	Freidora de Aire Philips	2	359,98
Electrodom.	Microondas Samsung	1	129,99

Interpretación de negocio: en Tecnología, Laptop e iPhone empatan en unidades (8) pero la Laptop genera un 18% más de ingresos: es el producto estrella a priorizar en stock y promoción. Destaca un patrón cruzado: en Hogar, el Sofá vende menos unidades que la Lámpara pero genera **casi 5× más ingresos**, así que las decisiones de surtido deben mirar volumen *e* ingresos por separado, no una sola métrica.

Reflexión comparativa

Ambos motores producen resultados idénticos, pero el camino difiere. **SQL** resulta más conciso y declarativo: el `JOIN` y las funciones ventana (`ROW_NUMBER`) expresan el problema en pocas líneas legibles, ideal para análisis tabulares. **MongoDB** exige pensar en etapas encadenadas (pipeline) y resolver el "top N por grupo" con la combinación `$push` + `$slice`, más verboso pero muy flexible con datos anidados o semiestructurados. En complejidad, SQL gana en estos análisis relacionales; en mantenibilidad, SQL es más directo para quien viene del modelo tabular, mientras Mongo brilla cuando los documentos ya vienen anidados y se evita el `$lookup`. La elección, por tanto, depende menos del "poder" de cada motor y más de cómo estén modelados los datos de origen.